

ASSIGNMENTS – OOADI

LECTURE 7

1.

- a) This one is easy ☺
- b) Well .. no it does not. The code is suppose to run 10 iterations, with each iteration launching 2 threads, each incrementing counter 100,000 times. The expected result should therefore be 200000 each time. However the code produces something like the following:

Iteration 0: Counter = 105984
Iteration 1: Counter = 200000
Iteration 2: Counter = 108124
Iteration 3: Counter = 200000

So sometimes it gets it right (200000) while it fails at other times. If you run it several times you will see that different results are seen. The reason for the failour is that the increment, implemented as 'counter = counter + 1', takes three steps at the CPU level, despite looking like one simple operation here. What actually happes is the following:

READ — load counter value from memory
ADD — add 1 to it
WRITE — store the result back

When two threads do this simultaneously:

Thread 1: READ counter (gets 50)
Thread 2: READ counter (gets 50) ← reads SAME value before T1 writes
Thread 1: ADD 50 + 1 = 51
Thread 2: ADD 50 + 1 = 51
Thread 1: WRITE counter = 51
Thread 2: WRITE counter = 51 ← overwrites T1, increment is LOST

As a consequence, one increment is silently lost. At 100,000 iterations per thread this can happen thousands of times.

- c) When trying to solve this issue we have a few options at hand. Two obvious ones are the following:

Option 1: Make use of the 'synchronized' keyword. This forces one thread at a time into increment(), thereby ensuring that one thread at a time can get access to the operation. This is a simple solution but it also creates a bottleneck.

```
static synchronized void increment() {  
    counter = counter + 1;  
}
```

Option 2: Make use of AtomicInteger (which is the recommended approach). This makes use of CPU-level atomic instructions and no locking is therefore needed.

```
static AtomicInteger counter = new AtomicInteger(0);  
static void increment() {  
    counter.incrementAndGet(); // atomic — cannot be interrupted  
}
```

d) Even easier .. especially if you just steal my example code ☺

-
2. a) To create a class specifically for handling user input we can implement something like:

```
public class CommandLineInputHandler implements Runnable {  
}
```

This then becomes a class dedicated solely to reading keyboard input from the user. It implements Runnable meaning that it is designed to run as a thread. It knows nothing about how the list works internally and it just calls methods on the manager. To implement a part which holds the list of trains in a separate thread we can do as follows:

```
public class TrainListManager {  
    private final List<Train> trains = new CopyOnWriteArrayList<>(  
        FleetData.sampleFleet()  
    );  
}
```

This class will then be dedicated solely to be responsible for owning and managing the train list. The 'CopyOnWriteArrayList' makes it thread-safe, which is critical since multiple threads will access it simultaneously.

Finally, to ensure that the program works with those two threads we need to revisit the AssignmentDemo, where we may add something like the following:

```
TrainListManager manager = new TrainListManager();  
  
Thread inputThread = new Thread(new CommandLineInputHandler(manager));  
inputThread.start();  
inputThread.join();
```

Here we set up 'manager' to run conceptually as the list-owning thread.

inputThread is the user input thread and join() ensures that main waits for the user to finish before proceeding. Both threads share the same manager object and the thread-safe list is the bridge between them.

Using an approach like the above we create a case where one thread handles user interaction, another owns the data, and they communicate safely through a thread-safe shared object rather than directly touching each other's internals.

b) Here, we add something like the following:

```
public class TrainOperator implements Runnable {  
}
```

This becomes a dedicated class that replaces the human keyboard input with automated actions. Also, by using 'implements Runnable' we again make sure that this can run as a thread, just like CommandLineInputHandler. Each instance represents one operator with its own identity and random number generator.

To emulate a few train operators we may add something like:

```
public TrainOperator(TrainListManager manager, String operatorId) {  
    this.manager = manager;  
    this.operatorId = operatorId;  
}
```

This ensure that each operator gets a unique ID, which allows us to see in the putput which operator did what. Again, they all share the same 'manager', which reflects a realistic case with operators all working on the same fleet. To provide the required activity options the TrainOperator class should include something like the following:

```
case 0 -> { // Search  
    manager.search(t -> t.getDriveType() == dt);  
}  
case 1 -> { // Insert  
    manager.addTrain(new DieselTrain("D-" + rng.nextInt(999), w));  
}  
case 2 -> { // Delete  
    manager.deleteTrain("D-" + rng.nextInt(999));  
}  
case 3 -> { // List all ← added for good measure 😊  
    manager.printAll();  
}
```

To make this run we need to update in AssignmentDemo again, to create and

launch the needed threads:

```
List<Thread> threads = new ArrayList<>();
for (int i = 0; i < 10; i++) {
    threads.add(new Thread(new TrainOperator(manager, "Operator-" + i)));
}
threads.forEach(Thread::start);
for (Thread t : threads) t.join();
```

Here we create 10 operator threads, where each run as a separate TrainOperator instance. The 'forEach(Thread::start)' launches all and they run concurrently. The 'join()' on each ensures that main waits for all operators to finish before we for instance choose to print the final fleet.

c) To automate the user input we return to TrainOperator again:

```
public class TrainOperator implements Runnable {
    private final Random rng = new Random();
```

Here, Random() replaces the scanner from CommandLineInputHandler, which allows the TrainOperator to generate its own decisions. All that has changed by this is the input source. To implement the randomness in operator action we can add something like this:

```
int action = rng.nextInt(4);
switch (action) {
    case 0 -> { // Search
    case 1 -> { // Insert
    case 2 -> { // Delete
    case 3 -> { // List all
```

The rng.nextInt(4) returns a random number between 0 and 3, mapping to each of the available operator actions. We would also use random within the actions themselves.

Emulating 10-100's of train operators is controlled in AssignmentDemo again:

```
for (int i = 0; i < 10; i++) { // ← change this number
    threads.add(new Thread(new TrainOperator(manager, "Operator-" + i)));
}
```

Notice that we again assign each operator a unique ID. Next we simulate that each operator runs 20 actions. To ensure "realistic" operation we add random timing between operator decisions using sleep():

```
for (int i = 0; i < 20; i++) { // random action
    Thread.sleep(rng.nextInt(100)); // random delay between actions
```

}

- d) Using `CopyOnWriteArrayList` we will eventually notice that the overall behaviour of the code changes depending on scale.

For thread counts of 10-ish the output will be clean and all operators finish predictably.

Increasing to 100-ish output risk becoming interleaved and hard to read. Operation is still expected to function okay. The problem here is related to the output as `System.out.println` is not atomic across multiple threads. So data will still be correct despite seeing jumbled output printed. This can be overcome by running the printing in a synchronized environment.

At 500-ish there will be a slowdown as garbage collection pressure starts to build.

At 1000+ we risk running into potential `OutOfMemoryError` and significant performance degradation.

The primary issue with `CopyOnWriteArrayList` is that it copies the entire list on each and every write operation. So while it is great for read-heavy workloads then it becomes very expensive for write-heavy operations. The fix could here be a `ReadWriteLock` as this would allow multiple operators to search simultaneously while writes still get exclusive access.
